Architecture.md v1.1

SUMMARY

Chapter 5 — Shared Memory Architecture

This chapter defines the memory system of AI OS.

The Memory Service is one of the most critical components in the entire architecture. It acts as the single source of truth for all persistent knowledge and context.

5.1 MEMORY PHILOSOPHY

AI OS shall maintain one unified memory.

There shall never be: - Jarvis Memory - Friday Memory - Analytics Memory - Business Memory

There is only: AI OS Memory

Every component reads from and writes to this centralized system through the Memory Service.

5.2 MEMORY GOALS

The Memory Service shall: - Store long-term user knowledge. - Maintain conversation continuity. - Preserve ongoing project information. - Provide contextual information to worker agents. - Support future semantic search. - Keep data synchronized across the entire system.

5.3 MEMORY LAYERS

The memory system is divided into five logical layers:

Layer 1 — Working Memory Purpose: Temporary context for the current request. Examples: Current conversation, Current task, Temporary calculations, Intermediate workflow data Lifetime: Only while the request is active. Automatically discarded afterward unless promoted.

Layer 2 — Session Memory Purpose: Maintain context during a conversation session. Examples: Current discussion topic, Recent questions, Active workflow Lifetime: Until the session ends.

Layer 3 — Long-Term Memory Purpose: Store information useful over months or years. Examples: User preferences, Frequently used workflows, Personal goals, Stable project information Only explicitly approved or high-confidence information is promoted here.

Layer 4 — Project Memory Purpose: Maintain context for each ongoing project. Example Project: AI OS Contains: Documents, Architecture, Decisions, Progress, Assigned tasks Another project should not access this memory unless explicitly requested.

Layer 5 — Knowledge Memory Purpose: Store reusable structured knowledge. Examples: Engineering formulas, Business templates, Coding conventions, Research summaries Knowledge is not tied to a single conversation.

5.4 MEMORY OWNERSHIP

Memory belongs to AI OS.

Not to: - Jarvis - Friday - Worker Agents - AI models

Replacing any component must not affect stored memory.

5.5 MEMORY ACCESS RULES

The following components may request memory: - Router (limited) - Jarvis - Friday - Agent Manager - Worker Agents

No component accesses the database directly. All access passes through the Memory Service.

5.6 MEMORY WRITE RULES

Only the Memory Service may persist information. Worker agents submit write requests.

Example flow: Analytics Agent → Memory Service → Database

Worker agents never write directly.

5.7 MEMORY CATEGORIES

Every stored item belongs to exactly one category.

User Profile: Name, Devices, Languages, Preferences Preferences examples: Favorite AI personality, Preferred coding language, Notification preferences Tasks examples: Pending, Completed, Scheduled, Cancelled Projects examples: AI OS, Anime Channel, Solar Cleaning Robot Conversations: Conversation summaries only; raw conversations should not be stored indefinitely. Files: Metadata only (File name, Tags, Project association) Research: Articles, notes, and findings linked to projects.

5.8 MEMORY LIFECYCLE

Every memory follows the same lifecycle: Created → Validated → Stored → Updated → Archived → Deleted (if approved)

Deletion of long-term memory requires explicit user approval.

5.9 MEMORY IMPORTANCE LEVELS

Every memory item has a priority.

Critical: Never delete automatically (User profile, API configuration, Active projects) Important: Retained long term (Goals, Preferences, Significant achievements) Normal: Can be archived (Meeting summaries, Temporary project notes) Temporary: Automatically expires (Current calculations, Temporary context)

5.10 CONTEXT RETRIEVAL

When an agent requests memory, the Memory Service returns only relevant information.

Example request: "Analyze Instagram performance." Returned context: Instagram account data, Previous analytics, Related project notes Not returned: Trading journal, College timetable, Business invoices

Context must remain focused.

5.11 MEMORY CONSISTENCY

Only one version of each fact should exist.

Example: Preferred editor: VS Code If updated: Cursor

The previous value is replaced or archived according to policy. Duplicate conflicting records are not permitted.

5.12 MEMORY SECURITY

Sensitive memories require elevated permissions. Examples: API keys, Financial records, Authentication tokens

These are encrypted and never exposed to worker agents unless explicitly authorized.

5.13 FUTURE MEMORY EXPANSION

Future versions may introduce: - Semantic search - Vector embeddings - Memory ranking - Confidence scoring - Automatic knowledge extraction - Memory compression - Cross-project linking

The architecture is designed to support these features without redesigning the Memory Service.

5.14 DESIGN RULES

The Memory Service must always satisfy: - Single source of truth. - Centralized persistence. - Minimal context retrieval. - No duplicate facts. - No direct database access from agents. - Secure handling of sensitive information. - Independent of AI model choice.

5.15 MEMORY ARCHITECTURE SUMMARY

Worker Agent → Memory Service → Memory Manager → Database → Vector Store (Future)

Every read and write operation must pass through this pipeline.

END OF CHAPTER 5

Next Chapter: Chapter 6 — Worker Agent Specification